Command Injection Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ OS Command Injection — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Test only in a local lab or explicitly authorized environment. Do not execute commands that read secrets, alter files, establish persistence, or affect production systems.*

■ Executive Summary

OS command injection occurs when untrusted input is incorporated into an operating-system command and interpreted as command syntax rather than ordinary data. The supplied WSTG-style document focuses on parameters that influence file retrieval and HTTP requests, where unsafe command construction can allow unintended process execution.

■ Core Testing Methodology

1. **Identify candidate inputs.** Review parameters that select files, documents, filenames, or other values likely to reach server-side process calls.
2. **Trace the normal request.** Record the expected request and response for a harmless test file.
3. **Use a controlled test environment.** Reproduce the relevant code path in staging or a local lab before testing any command-injection hypothesis.
4. **Use non-destructive indicators.** Prefer application logs, mocked process calls, or benign fixed test markers rather than commands that access system files or execute arbitrary programs.
5. **Check input handling.** Verify that delimiters, encoded values, and unexpected characters are rejected or treated as literal data.
6. **Review POST and query parameters.** Determine whether different request locations reach the same unsafe process invocation.
7. **Verify server-side impact.** Do not infer command execution from a changed status code alone; use controlled, auditable evidence.
8. **Document the code path.** Record the parameter, expected behavior, observed behavior, and relevant safe evidence.
9. **Retest remediation.** Confirm the vulnerable process call has been removed or made safe.

■ High-Value Bug Bounty Tips & Edge Cases

- **Filename parameters:** File paths and document identifiers are common places to review, but a parameter alone does not prove shell execution.
- **Encoded delimiters:** URL decoding and multiple parsing layers can produce inconsistent validation. Validate the canonical value and avoid repeated decoding.
- **POST body inputs:** The source includes a POST request example involving document retrieval. Check whether body fields reach a shell command.
- **Unsafe concatenation:** Any use of shell command strings with user-controlled input deserves review.
- **Error behavior:** Errors may reveal implementation details but do not alone prove command injection.
- **Source limitation:** The PDF includes legacy examples and does not provide a modern, safe lab or verified result for a specific target.

■■ Educational Checklists & Safe Examples

Example of a safe, non-executable input-validation test:

```
POST /documents/preview HTTP/1.1
Host: example.com
```

```
Content-Type: application/x-www-form-urlencoded

document_id=sample-document-001
```

- [ ] Inputs are treated as data, not shell syntax.
- [ ] The application uses safe APIs rather than shell-string concatenation.
- [ ] File access is constrained to an approved directory.
- [ ] Invalid values fail safely.
- [ ] Logs and errors do not expose secrets or system details.

■■ Mitigation & Remediation

- Avoid invoking a shell where a language-native API can perform the task.
- Use fixed executable paths and argument arrays rather than concatenated command strings.
- Validate identifiers against strict allowlists and map them to server-controlled resources.
- Apply least privilege and isolate processes.
- Restrict file access and execution permissions.
- Log failures safely and add regression tests for malformed and unexpected input.

■ Source Notes

Based on the supplied five-page *Command injection.pdf*, which identifies itself as a WSTG-style “Testing for Command Injection” document. Page 1 explains the issue and shows URL-based file selection examples; page 2 shows a POST-based document retrieval request and a modified-input example. The remaining pages contain little readable text in the supplied preview, so no further source-specific details are inferred.

Educational use only. Never run destructive commands or test outside authorization.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.